

Downfall: PPO Trained Agent for a 4 Stage Godot Obstacle Course

This project trains a reinforcement learning (RL) agent to play Downfall, a 3D obstacle course game inspired by Fall Guys, built with the Godot engine. The game has four stages that must be cleared in a single run. The agent is trained with PPO (Proximal Policy Optimization) and acts without human input once training is complete.

The agent receives information about the environment using raycasts from Godot and outputs continuous control actions for movement and jumping.

Github Repo Link

- [Click here for the github repo](#)
-

Contributors

- Aman Kumar Prajapati (22BEC006)
 - Arjit Verma (22BCS015)
 - Nabeel Ahsan (22BEC026)
 - Arsalan Shaik (22BDS053)
 - Subburi Dheeraj Verma (22BCS125)
-

Table of contents

1. Project overview
2. Task description
3. Environment and observations
4. Training data (interaction data)
5. Steps to build and train the RL model
6. Reward model
7. Training logs
8. Accuracy testing and evaluation protocol
9. How to run the project

1. Project overview

- Engine: Godot (3D project)
- Game name: Downfall
- Stages: 4 consecutive stages in one episode
- Control: RL agent controls the player character
- Algorithm: PPO from Stable Baselines3
- Observation source: Raycasts and high level state from Godot
- Action space: Continuous movement, rotation and jump

Goal: build an agent that reliably completes all four stages only from interaction and numeric rewards, without hard coded paths.

2. Task description

The problem is a navigation and survival task in a four stage obstacle course.

- The agent controls a player in a 3D level with:
 - Static and moving platforms and tiles
 - Gaps and holes
 - Hazards such as bombs and moving obstacles
 - A final goal area at the end of stage 4
- The game is divided into four stages. When one stage is completed the player is moved to the next stage. An episode ends when either:
 - The agent reaches the final goal after stage 4 (success), or
 - The agent falls off the map, triggers a failure condition, or the time limit is reached (failure)

Episodes are repeated many times during training. The agent improves by trying to reach the goal more often and in fewer steps.

3. Environment and observations

3.1 Godot scene and scripts

Key Godot pieces used in this project:

- `player.gd`
 - Extends `CharacterBody3D`
 - Applies movement, jumping, gravity and basic animation
 - Communicates with the RL bridge script
 - Maintains reward related signals such as progress and failure
- `ai_controller.gd` (name may differ in your project)
 - Bridge between Godot and the Python training process
 - Exposes functions such as:
 - `get_obs()` builds the observation vector
 - `get_obs_space()` and `get_action_space()` describe observation and action sizes
 - `set_action(action)` applies agent actions to the player
 - `get_reward()` returns numeric reward since last step and resets it
 - `reset()` resets the environment to the start of stage 1 and clears internal counters
- Hazard and goal scripts such as `bomb.gd`, `falling_tile.gd`, `spike_roller.gd`, `swiper.gd`, `end_gate.gd`, `spawn_box.gd`
 - Mark the player as dead when hit or when falling below a safe height
 - Mark stages as completed when the player reaches the gate or goal trigger
 - Inform `player.gd` so that reward and episode termination flags can be updated

All four stages are built inside the Godot scene. Stage index is tracked so the reward and observations can depend on it.

3.2 Observations from raycasts

The agent does not see the whole game state. Instead, `ai_controller.gd` builds an observation vector from a limited set of signals:

- Raycasts:
 - Several rays are cast from the player character in different directions: forward, backward, left, right and downward
 - For each ray we record:
 - Normalized distance to the first object hit
 - Simple type code for the object (ground, wall, hazard) encoded as a number
- Goal related information:
 - Vector from the player to the current stage goal or final gate (direction and distance)

- Stage index (1 to 4) or a set of four values where the current stage has value 1 and the others have value 0
- Player state:
 - On ground flag
 - Current vertical velocity
 - Optional high level flags, for example whether the player is currently jumping

All these values are concatenated into a single numeric vector returned by `get_obs()` at every decision step.

3.3 Action space

The PPO policy outputs a continuous action vector, interpreted inside `set_action(action)` as:

- Forward or backward movement value
- Left or right movement or rotation value
- Jump value (continuous output converted to jump or no jump using a threshold)

Actions are applied each physics step in Godot. There is no built in script that tells the agent what to do. The policy learns which action values work best through the reward model described later.

4. Training data (interaction data)

There is no static offline dataset. Data is created while the agent interacts with the game.

At each step the following tuple is produced:

- `state_t` : observation vector from `get_obs()`
- `action_t` : action chosen by the policy network
- `reward_t` : scalar reward computed inside Godot and returned by `get_reward()`
- `state_t_plus_1` : next observation
- `done_t` : boolean that is true if the episode ended at this step

These tuples are stored in memory in the PPO implementation. After a fixed number of steps, PPO uses this stored data to update the policy network and the value network.

The total size of the training data depends on the configured number of steps and how long training runs. With more time the agent sees more failures and successes across all four stages, which helps it learn more robust behavior.

5. Steps to build and train the RL model

This section describes the training pipeline from the Godot project to a trained PPO model.

5.1 Prepare the Godot project

1. Open the Downfall project in Godot.
2. Confirm that:
 - All four stages are connected so that the player moves from stage 1 to stage 4 in one episode
 - `player.gd` and `ai_controller.gd` are attached to the correct nodes
 - Hazard and gate scripts are connected to the correct signals
3. Implement and test the methods needed for RL:
 - `get_obs()` returns a stable numeric vector
 - `set_action(action)` moves the player according to the action
 - `get_reward()` returns reward and resets the internal reward accumulator
 - `reset()` resets stage, position, velocities, and internal flags
4. Export the Godot project to a standalone binary that the Python script can start, for example:
 - `Downfall.x86_64` on Linux
 - `Downfall.exe` on Windows

5.2 Wrap the Godot environment in Python

We use a Stable Baselines compatible wrapper:

```
from godot_rl.wrappers.stable_baselines_wrapper import StableBaselinesGodotEnv
from stable_baselines3.common.vec_env import VecMonitor

env = StableBaselinesGodotEnv(
    env_path="path/to/Downfall.x86_64",
    show_window=False,          # set True to see the game while training
    seed=0,
    n_parallel=8,               # run 8 game instances in parallel
    speedup=5.0,               # speed multiplier inside Godot
)

env = VecMonitor(env)
```

`VecMonitor` records episode reward and length for later analysis.

5.3 Define the PPO agent

```
from stable_baselines3 import PPO

model = PPO(
    policy="MultiInputPolicy",
    env=env,
    learning_rate=0.0003,
    n_steps=32,
    batch_size=256,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    ent_coef=0.0001,
    verbose=2,
    tensorboard_log="logs/sb3",
)
```

Explanation of key settings in simple terms:

- **policy** : uses a neural network that can handle the observation format produced by Godot
- **learning_rate** : size of parameter updates. A smaller value gives slower but usually more stable learning
- **n_steps** : number of steps collected per environment before doing a training update
- **batch_size** : number of samples used in each gradient step during learning
- **gamma** : how much the agent cares about future reward compared to immediate reward
- **clip_range** : bounds how much the new policy is allowed to change compared to the old one in a single update

5.4 Run training

Example training script:

```
total_timesteps = 1_000_000

model.learn(
    total_timesteps=total_timesteps,
    tb_log_name="downfall_ppo",
)

model.save("models/downfall_ppo.zip")
```

`total_timesteps` is the total number of environment steps seen across all parallel instances. For example, with 8 parallel environments and `n_steps = 32`, each update uses $8 * 32 = 256$ steps.

6. Reward model

The reward model is implemented on the Godot side in `player.gd` and related scripts. Its goal is to guide the agent toward safe and efficient completion of the four stages.

Typical reward components are:

1. Progress reward

- At each step compute the distance from the player to the current stage goal.
- If the player moves closer to the goal since the previous step, give a small positive reward.
- If the player moves away from the goal, give a small negative reward.

2. Time penalty

- Give a small negative reward at every step.
- This discourages the agent from standing still or wandering without progress.

3. Stage completion reward

- When the agent reaches the end gate of a stage, give a large positive reward.
- When the agent clears stage 4 and reaches the final goal, give the largest positive reward and mark the episode as success.

4. Failure penalties

- If the player:
 - Falls below a safe vertical threshold
 - Is hit by a bomb or similar hazard
 - Gets stuck until time limit is reached
- Then:
 - Give a large negative reward
 - Mark the episode as done

Implementation pattern:

- `player.gd` keeps a variable such as `current_reward`.
- Hazard or goal scripts call methods on the player to add or subtract from `current_reward` and set flags for death or success.

- `ai_controller.gd`:
 - Returns `current_reward` in `get_reward()` and then resets it to zero.
 - Uses flags to compute `done` and to decide when to call `reset()`.

This design makes the reward logic easy to change without modifying the Python training code.

7. Training logs

Training progression is recorded using TensorBoard.

When `tensorboard_log="logs/sb3"` is set in the PPO constructor, running `model.learn(...)` will create log files under `logs/sb3/downfall_ppo`.

Logged metrics include:

- Mean reward per episode
- Mean episode length
- Policy loss
- Value loss

Training logs

- [Training logs video](#)
-

8. Accuracy testing and evaluation protocol

After training, we measure how well the agent plays the game. This section describes a simple evaluation procedure.

8.1 Evaluation script

```
import numpy as np
from godot_rl.wrappers.stable_baselines_wrapper import StableBaselinesGodot
from stable_baselines3 import PPO

model = PPO.load("models/downfall_ppo.zip")

env_eval = StableBaselinesGodotEnv(
    env_path="path/to/Downfall.x86_64",
```



```

        show_window=True,
        seed=123,
        n_parallel=1,
        speedup=1.0,
    )

n_eval_episodes = 100
episode_rewards = []
successes = 0

for ep in range(n_eval_episodes):
    obs = env_eval.reset()
    done = [False]
    total_reward = 0.0
    success = False

    while not done[0]:
        action, _ = model.predict(obs, deterministic=True)
        obs, reward, done, info = env_eval.step(action)
        total_reward += reward[0]

        # If the environment sets a "success" flag in info:
        if info[0].get("success", False):
            success = True

    episode_rewards.append(total_reward)
    if success:
        successes += 1

mean_reward = float(np.mean(episode_rewards))
std_reward = float(np.std(episode_rewards))
success_rate = successes / n_eval_episodes

print("Mean reward per episode:", mean_reward)
print("Reward standard deviation:", std_reward)
print("Success rate:", success_rate)

```

9. How to run the project

9.1 Requirements

- Godot engine version used for the game

- Python 3
- Python packages:
 - `stable-baselines3`
 - `godot-rl` or the Godot RL wrapper used in this repo
 - `tensorboard`
 - `numpy`

Install dependencies:

```
pip install -r requirements.txt
```

9.2 Train a new agent

Example command:

```
python train_downfall_ppo.py --env_path path/to/Downfall.x86_64 --exp
```

Adjust `--timesteps` depending on how much training you want to perform.

9.3 Run a trained agent

```
python train_downfall_ppo.py --env_path path/to/Downfall.x86_64 --load
```

This will load the saved PPO agent, run it in the Godot environment and display the four stages in a game window.

10. Media (videos of the output)

10.1 Training and Results video

Replace each placeholder link with your actual video URLs.

- Training logs
 - [Training logs video](#)
- Training process run (video of the environment while PPO is learning)
 - [Training process video](#)

- Trained agent inference (final agent playing all four stages)
 - [Trained agent inference video](#)